

Punto 4.8 - Split Queries: cuándo y cómo usarlas

Práctica

Ejercicio: Añadir métodos al repositorio del proyecto AceriaData que demuestren el producto cartesiano, AsSplitQuery y AsSingleQuery. Comparar el número de consultas, el volumen de datos y el tiempo de ejecución. Configurar el comportamiento por defecto y la advertencia de producto cartesiano.

Contexto del proyecto: En el punto 4.7 se estudiaron las consultas ineficientes, incluyendo los filtros no traducibles y las funciones en Where. En este punto se profundiza en las Split Queries, que permiten evitar el producto cartesiano cuando se incluyen varias colecciones. Esta técnica se usará en el punto 4.9 para las Compiled Queries.

Paso 1: Abrir el proyecto

```
cd AceriaData
cd src/AceriaData.Console
```

cd AceriaData → entra en la carpeta raíz del proyecto. cd src/AceriaData.Console → entra en la carpeta del proyecto de consola.

Paso 2: Añadir los métodos de Split Queries a la interfaz del repositorio

Modificar src/AceriaData.Application/Interfaces/IOrdenRepositorio.cs:

```
using AceriaData.Application.Dtos;
using AceriaData.Domain.Entities;

namespace AceriaData.Application.Interfaces;

public interface IOrdenRepositorio
{
    // ... métodos existentes ...

    List<OrdenFabricacion> ObtenerConVariasColeccionesSingleQuery();
    List<OrdenFabricacion> ObtenerConVariasColeccionesSplitQuery();
    string ObtenerSqlSingleQuery();
    string ObtenerSqlSplitQuery();

    void Agregar(OrdenFabricacion orden);
    void Eliminar(OrdenFabricacion orden);
}
```

Línea 1: using AceriaData.Application.Dtos; → importa los DTOs.

Línea 2: using AceriaData.Domain.Entities; → importa las entidades.

Línea 4: namespace AceriaData.Application.Interfaces; → declara el espacio de nombres.

Línea 6: public interface IOrdenRepositorio → declara la interfaz.

Línea 8: // ... métodos existentes ... → comentario que indica que se mantienen los métodos anteriores.

Línea 10: List ObtenerConVariasColeccionesSingleQuery(); → método que usa AsSingleQuery.

Línea 11: List ObtenerConVariasColeccionesSplitQuery(); → método que usa AsSplitQuery.

Línea 12: string ObtenerSqlSingleQuery(); → método que devuelve el SQL de AsSingleQuery.

Línea 13: string ObtenerSqlSplitQuery(); → método que devuelve el SQL de AsSplitQuery.

Línea 15: void Agregar(OrdenFabricacion orden); → método que agrega una orden.

Línea 16: void Eliminar(OrdenFabricacion orden); → método que elimina una orden.

Error común: si los métodos devuelven IQueryable, la capa de aplicación puede componer consultas que el repositorio no controla. Se deben devolver listas o valores concretos.

| Paso 3: Implementar los métodos de Split Queries en el repositorio

```

Modificar src/AceriaData.Infrastructure/Repositories/OrdenRepositorio.cs: \csharp public List
ObtenerConVariasColeccionesSingleQuery() { return
_context.OrdenesFabricacion .AsNoTrackingWithIdentityResolution() .Include(o => o.Planchas) .Include(o
=> o.OrdenesAleaciones) .ThenInclude(oa => oa.Aleacion) .AsSingleQuery() .OrderBy(o =>
o.NumeroOrden) .ToList(); }
    
```

```

public List ObtenerConVariasColeccionesSplitQuery() { return
_context.OrdenesFabricacion .AsNoTrackingWithIdentityResolution() .Include(o => o.Planchas) .Include(o
=> o.OrdenesAleaciones) .ThenInclude(oa => oa.Aleacion) .AsSplitQuery() .OrderBy(o =>
o.NumeroOrden) .ToList(); }
    
```

```

public string ObtenerSqlSingleQuery() { var consulta =
_context.OrdenesFabricacion .AsNoTracking() .Include(o => o.Planchas) .Include(o =>
o.OrdenesAleaciones) .AsSingleQuery();
    
```

```
return consulta.ToQueryString();
```

```
}
```

```

public string ObtenerSqlSplitQuery() { var consulta =
_context.OrdenesFabricacion .AsNoTracking() .Include(o => o.Planchas) .Include(o =>
o.OrdenesAleaciones) .AsSplitQuery();
    
```

```
return consulta.ToQueryString();
```

} Línea 1: public List ObtenerConVariasColeccionesSingleQuery() → declara el método.

Línea 3: return _context.OrdenesFabricacion → inicia la consulta.

Línea 4: .AsNoTrackingWithIdentityResolution() → aplica resolución de identidad.

Línea 5: .Include(o => o.Planchas) → incluye la colección de planchas.

Línea 6: .Include(o => o.OrdenesAleaciones) → incluye la colección de entidades intermedias.

Línea 7: .ThenInclude(oa => oa.Aleacion) → incluye la aleación de cada entidad intermedia.

Línea 8: .AsSingleQuery() → fuerza una sola consulta.

Línea 9: .OrderBy(o => o.NumeroOrden) → ordena por número de orden.

Línea 10: .ToList(); → materializa la consulta.

Línea 13: public List ObtenerConVariasColeccionesSplitQuery() → declara el método.

Línea 15: return _context.OrdenesFabricacion → inicia la consulta.

Línea 16: .AsNoTrackingWithIdentityResolution() → aplica resolución de identidad.

Línea 17: .Include(o => o.Planchas) → incluye la colección de planchas.

Línea 18: .Include(o => o.OrdenesAleaciones) → incluye la colección de entidades intermedias.

Línea 19: .ThenInclude(oa => oa.Aleacion) → incluye la aleación de cada entidad intermedia.

Línea 20: .AsSplitQuery() → divide la consulta en varias.

Línea 21: .OrderBy(o => o.NumeroOrden) → ordena por número de orden.

Línea 22: .ToList(); → materializa la consulta.

Línea 25: `public string ObtenerSqlSingleQuery()` → declara el método.

Línea 27: `var consulta = _context.OrdenesFabricacion` → inicia la consulta.

Línea 28: `.AsNoTracking()` → aplica `AsNoTracking`.

Línea 29: `.Include(o => o.Planchas)` → incluye la colección de planchas.

Línea 30: `.Include(o => o.OrdenesAleaciones)` → incluye la colección de entidades intermedias.

Línea 31: `.AsSingleQuery();` → fuerza una sola consulta.

Línea 33: `return consulta.ToQueryString();` → devuelve el SQL.

Línea 36: `public string ObtenerSqlSplitQuery()` → declara el método.

Línea 38: `var consulta = _context.OrdenesFabricacion` → inicia la consulta.

Línea 39: `.AsNoTracking()` → aplica `AsNoTracking`.

Línea 40: `.Include(o => o.Planchas)` → incluye la colección de planchas.

Línea 41: `.Include(o => o.OrdenesAleaciones)` → incluye la colección de entidades intermedias.

Línea 42: `.AsSplitQuery();` → divide la consulta en varias.

Línea 44: `return consulta.ToQueryString();` → devuelve el SQL.

Error común: si se usa `AsSplitQuery` con una sola colección, no aporta beneficio porque no hay producto cartesiano. Se debe usar solo con varias colecciones.

| Paso 4: Crear el caso de uso de Split Queries

Crear el archivo `src/AceriaData.Application/UseCases/SplitQueriesUseCase.cs`:

```
using System.Diagnostics;
using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public class SplitQueriesUseCase
{
    private readonly IUnidadDeTrabajo _unidad;

    public SplitQueriesUseCase(IUnidadDeTrabajo unidad)
    {
        _unidad = unidad;
    }

    public void Ejecutar()
    {
        Console.WriteLine("=== SPLIT QUERIES ===");

        DemostrarSingleQuery();
        DemostrarSplitQuery();
        CompararRendimiento();
        MostrarSql();
    }

    private void DemostrarSingleQuery()
    {
        Console.WriteLine("\n-- Single Query (producto cartesiano) --");

        var cronometro = Stopwatch.StartNew();
        var ordenes = _unidad.Ordenes.ObtenerConVariasColeccionesSingleQuery();
        cronometro.Stop();

        Console.WriteLine($"Órdenes: {ordenes.Count} | Tiempo: {cronometro.ElapsedMilliseconds} ms");
        foreach (var orden in ordenes)
        {
            Console.WriteLine($"  {orden.NumeroOrden}: {orden.Planchas.Count} planchas |
```

```

{orden.OrdenesAleaciones.Count} aleaciones");
    }
}

private void DemostrarSplitQuery()
{
    Console.WriteLine("\n--- Split Query (sin producto cartesiano) ---");

    var cronometro = Stopwatch.StartNew();
    var ordenes = _unidad.Ordenes.ObtenerConVariasColeccionesSplitQuery();
    cronometro.Stop();

    Console.WriteLine($"Órdenes: {ordenes.Count} | Tiempo: {cronometro.ElapsedMilliseconds} ms");
    foreach (var orden in ordenes)
    {
        Console.WriteLine($" {orden.NumeroOrden}: {orden.Planchas.Count} planchas |
{orden.OrdenesAleaciones.Count} aleaciones");
    }
}

private void CompararRendimiento()
{
    Console.WriteLine("\n--- Comparación de rendimiento ---");

    var cronometroSingle = Stopwatch.StartNew();
    var ordenesSingle = _unidad.Ordenes.ObtenerConVariasColeccionesSingleQuery();
    cronometroSingle.Stop();

    var cronometroSplit = Stopwatch.StartNew();
    var ordenesSplit = _unidad.Ordenes.ObtenerConVariasColeccionesSplitQuery();
    cronometroSplit.Stop();

    Console.WriteLine($"Single Query: {cronometroSingle.ElapsedMilliseconds} ms | Órdenes:
{ordenesSingle.Count}");
    Console.WriteLine($"Split Query: {cronometroSplit.ElapsedMilliseconds} ms | Órdenes:
{ordenesSplit.Count}");
}

private void MostrarSql()
{
    Console.WriteLine("\n--- SQL de Single Query ---");
    var sqlSingle = _unidad.Ordenes.ObtenerSqlSingleQuery();
    Console.WriteLine(sqlSingle);

    Console.WriteLine("\n--- SQL de Split Query ---");
    var sqlSplit = _unidad.Ordenes.ObtenerSqlSplitQuery();
    Console.WriteLine(sqlSplit);
}
}

```

Línea 1: using System.Diagnostics; → importa Stopwatch.

Línea 2: using AceriaData.Application.Interfaces; → importa las interfaces de la aplicación.

Línea 4: namespace AceriaData.Application.UseCases; → declara el espacio de nombres.

Línea 6: public class SplitQueriesUseCase → declara el caso de uso.

Línea 8: private readonly IUnidadDeTrabajo _unidad; → campo de la unidad de trabajo.

Línea 10: public SplitQueriesUseCase(IUnidadDeTrabajo unidad) → constructor.

Línea 12: _unidad = unidad; → asigna el parámetro al campo.

Línea 15: public void Ejecutar() → declara el método principal.

Línea 17: Console.WriteLine("=== SPLIT QUERIES ==="); → muestra la cabecera.

Línea 19: DemostrarSingleQuery(); → llama al método de SingleQuery.

Línea 20: DemostrarSplitQuery(); → llama al método de SplitQuery.

Línea 21: CompararRendimiento(); → llama al método de comparación.

Línea 22: MostrarSql(); → llama al método de SQL.

Línea 25: private void DemostrarSingleQuery() → declara el método.

Línea 27: Console.WriteLine("\n--- Single Query (producto cartesiano) ---"); → muestra la cabecera.

Línea 29: var cronometro = Stopwatch.StartNew(); → inicia el cronómetro.

Línea 30: var ordenes = _unidad.Ordenes.ObtenerConVariacionesSingleQuery(); → carga las órdenes con AsSingleQuery.

Línea 31: cronometro.Stop(); → detiene el cronómetro.

Línea 33: Console.WriteLine(\$"Órdenes: {ordenes.Count} | Tiempo: {cronometro.ElapsedMilliseconds} ms"); → muestra el número de órdenes y el tiempo.

Línea 34: foreach (var orden in ordenes) → itera sobre las órdenes.

Línea 36: Console.WriteLine(\$" {orden.NumeroOrden}: {orden.Planchas.Count} planchas | {orden.OrdenesAleaciones.Count} aleaciones"); → muestra los datos.

Línea 39: private void DemostrarSplitQuery() → declara el método.

Línea 41: Console.WriteLine("\n--- Split Query (sin producto cartesiano) ---"); → muestra la cabecera.

Línea 43: var cronometro = Stopwatch.StartNew(); → inicia el cronómetro.

Línea 44: var ordenes = _unidad.Ordenes.ObtenerConVariacionesSplitQuery(); → carga las órdenes con AsSplitQuery.

Línea 45: cronometro.Stop(); → detiene el cronómetro.

Línea 47: Console.WriteLine(\$"Órdenes: {ordenes.Count} | Tiempo: {cronometro.ElapsedMilliseconds} ms"); → muestra el número de órdenes y el tiempo.

Línea 48: foreach (var orden in ordenes) → itera sobre las órdenes.

Línea 50: Console.WriteLine(\$" {orden.NumeroOrden}: {orden.Planchas.Count} planchas | {orden.OrdenesAleaciones.Count} aleaciones"); → muestra los datos.

Línea 54: private void CompararRendimiento() → declara el método.

Línea 56: Console.WriteLine("\n--- Comparación de rendimiento ---"); → muestra la cabecera.

Línea 58: var cronometroSingle = Stopwatch.StartNew(); → inicia el cronómetro.

Línea 59: var ordenesSingle = _unidad.Ordenes.ObtenerConVariacionesSingleQuery(); → carga las órdenes con AsSingleQuery.

Línea 60: cronometroSingle.Stop(); → detiene el cronómetro.

Línea 62: var cronometroSplit = Stopwatch.StartNew(); → inicia el cronómetro.

Línea 63: var ordenesSplit = _unidad.Ordenes.ObtenerConVariacionesSplitQuery(); → carga las órdenes con AsSplitQuery.

Línea 64: cronometroSplit.Stop(); → detiene el cronómetro.

Línea 66: Console.WriteLine(\$"Single Query: {cronometroSingle.ElapsedMilliseconds} ms | Órdenes: {ordenesSingle.Count}"); → muestra el tiempo de SingleQuery.

Línea 67: Console.WriteLine(\$"Split Query: {cronometroSplit.ElapsedMilliseconds} ms | Órdenes: {ordenesSplit.Count}"); → muestra el tiempo de SplitQuery.

Línea 70: private void MostrarSql() → declara el método.

Línea 72: Console.WriteLine("\n--- SQL de Single Query ---"); → muestra la cabecera.

Línea 73: `var sqlSingle = _unidad Ordenes.ObtenerSqlSingleQuery();` → obtiene el SQL.

Línea 74: `Console.WriteLine(sqlSingle);` → imprime el SQL.

Línea 76: `Console.WriteLine("\n--- SQL de Split Query ---");` → muestra la cabecera.

Línea 77: `var sqlSplit = _unidad Ordenes.ObtenerSqlSplitQuery();` → obtiene el SQL.

Línea 78: `Console.WriteLine(sqlSplit);` → imprime el SQL.

Error común: si se ejecutan las dos consultas en el mismo DbContext, las entidades de la primera consulta permanecen en el Change Tracker. Se debe usar un DbContext distinto para cada consulta si se quiere medir el tiempo de forma aislada.

| Paso 5: Registrar el caso de uso en el contenedor

Modificar `src/AceriaData.Console/Program.cs`:

```
services.AddScoped<SplitQueriesUseCase>();
```

Línea 1: `services.AddScoped();` → registra el caso de uso con ciclo de vida Scoped.

Error común: si se registra con Singleton, se comparte la misma instancia del DbContext entre todos los ámbitos y se producen problemas de concurrencia.

| Paso 6: Llamar al caso de uso desde la consola

Modificar el método Main:

```
using (var scope = _provider.CreateScope())
{
    var useCase = scope.ServiceProvider.GetRequiredService<SplitQueriesUseCase>();
    useCase.Ejecutar();
}
```

Línea 1: `using (var scope = _provider.CreateScope())` → crea un ámbito.

Línea 3: `var useCase = scope.ServiceProvider.GetRequiredService();` → resuelve el caso de uso.

Línea 4: `useCase.Ejecutar();` → ejecuta el caso de uso.

Error común: si se resuelve el caso de uso desde el proveedor raíz en lugar de desde un ámbito, se lanza una excepción `InvalidOperationException`.

| Paso 7: Insertar datos de prueba con varias planchas y aleaciones

Asegurarse de que hay datos en la base de datos. Modificar el método Main para insertar órdenes, planchas y aleaciones:

```
using (var scope = _provider.CreateScope())
{
    var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
    context.Database.EnsureDeleted();
    context.Database.Migrate();

    var aleacion1 = new Aleacion { Nombre = "AISI 1045",Codigo = "A1045", PorcentajeCarbono = 0.45m,
    PorcentajeManganeso = 0.75m, Activo = true };
    var aleacion2 = new Aleacion { Nombre = "AISI 4140",Codigo = "A4140", PorcentajeCarbono = 0.40m,
    PorcentajeManganeso = 0.85m, Activo = true };
    context.Aleaciones.AddRange(aleacion1, aleacion2);
    context.SaveChanges();

    var orden1 = new OrdenFabricacion { NumeroOrden = "OF-2024-0001", Cliente = "Constructora del
    Norte", Estado = "Pendiente", FechaCreacion = new DateTime(2024, 1, 15) };
    var orden2 = new OrdenFabricacion { NumeroOrden = "OF-2024-0002", Cliente = "Constructora del Sur",
    Estado = "Pendiente", FechaCreacion = new DateTime(2024, 2, 20) };
```

```

var orden3 = new OrdenFabricacion { NumeroOrden = "OF-2024-0003", Cliente = "Constructora del
Este", Estado = "EnProceso", FechaCreacion = new DateTime(2024, 3, 10) };
context.OrdenesFabricacion.AddRange(orden1, orden2, orden3);
context.SaveChanges();

var planchas = new List<PlanchaAcero>
{
    new PlanchaAcero { OrdenId = orden1.Id, Espesor = 10.5, Ancho = 1500, Largo = 3000, Peso =
370.5m, Activa = true },
    new PlanchaAcero { OrdenId = orden1.Id, Espesor = 12.0, Ancho = 1200, Largo = 2500, Peso =
280.8m, Activa = true },
    new PlanchaAcero { OrdenId = orden1.Id, Espesor = 9.0, Ancho = 1100, Largo = 2200, Peso =
180.0m, Activa = true },
    new PlanchaAcero { OrdenId = orden2.Id, Espesor = 8.0, Ancho = 1000, Largo = 2000, Peso =
125.6m, Activa = true },
    new PlanchaAcero { OrdenId = orden2.Id, Espesor = 11.0, Ancho = 1300, Largo = 2700, Peso =
290.0m, Activa = true },
    new PlanchaAcero { OrdenId = orden3.Id, Espesor = 15.0, Ancho = 1800, Largo = 3500, Peso =
450.0m, Activa = true }
};
context.PlanchasAcero.AddRange(planchas);

var ordenAleaciones = new List<OrdenAleacion>
{
    new OrdenAleacion { OrdenFabricacionId = orden1.Id, AleacionId = aleacion1.Id,
CantidadUtilizada = 1500.5m, EstadoRelacion = "Activa" },
    new OrdenAleacion { OrdenFabricacionId = orden1.Id, AleacionId = aleacion2.Id,
CantidadUtilizada = 800.0m, EstadoRelacion = "Activa" },
    new OrdenAleacion { OrdenFabricacionId = orden2.Id, AleacionId = aleacion1.Id,
CantidadUtilizada = 1200.0m, EstadoRelacion = "Activa" },
    new OrdenAleacion { OrdenFabricacionId = orden3.Id, AleacionId = aleacion2.Id,
CantidadUtilizada = 900.0m, EstadoRelacion = "Activa" }
};
context.OrdenesAleaciones.AddRange(ordenAleaciones);

context.SaveChanges();
}

```

Línea 1: using (var scope = _provider.CreateScope()) → crea un ámbito.

Línea 3: var context = scope.ServiceProvider.GetRequiredService(); → resuelve el DbContext.

Línea 4: context.Database.EnsureDeleted(); → elimina la base de datos.

Línea 5: context.Database.Migrate(); → aplica las migraciones.

Línea 7: var aleacion1 = new Aleacion { ... }; → crea la primera aleación.

Línea 8: var aleacion2 = new Aleacion { ... }; → crea la segunda aleación.

Línea 9: context.Aleaciones.AddRange(aleacion1, aleacion2); → registra las aleaciones.

Línea 10: context.SaveChanges(); → inserta las aleaciones.

Línea 12: var orden1 = new OrdenFabricacion { ... }; → crea la primera orden.

Línea 13: var orden2 = new OrdenFabricacion { ... }; → crea la segunda orden.

Línea 14: var orden3 = new OrdenFabricacion { ... }; → crea la tercera orden.

Línea 15: context.OrdenesFabricacion.AddRange(orden1, orden2, orden3); → registra las órdenes.

Línea 16: context.SaveChanges(); → inserta las órdenes.

Línea 18: var planchas = new List → crea la lista de planchas.

Línea 20: new PlanchaAcero { ... }, → primera plancha.

Línea 21: new PlanchaAcero { ... }, → segunda plancha.

Línea 22: new PlanchaAcero { ... }, → tercera plancha.

Línea 23: new PlanchaAcero { ... }, → cuarta plancha.

Línea 24: new PlanchaAcero { ... }, → quinta plancha.

Línea 25: new PlanchaAcero { ... } → sexta plancha.

Línea 26: }; → cierra la lista.

Línea 27: context.PlanchasAcero.AddRange(planchas); → registra las planchas.

Línea 29: var ordenAleaciones = new List → crea la lista de relaciones.

Línea 31: new OrdenAleacion { ... }, → primera relación.

Línea 32: new OrdenAleacion { ... }, → segunda relación.

Línea 33: new OrdenAleacion { ... }, → tercera relación.

Línea 34: new OrdenAleacion { ... } → cuarta relación.

Línea 35: }; → cierra la lista.

Línea 36: context OrdenesAleaciones.AddRange(ordenAleaciones); → registra las relaciones.

Línea 38: context.SaveChanges(); → inserta las planchas y las relaciones.

Error común: si se insertan las planchas antes que las órdenes, la clave foránea OrdenId no tiene un valor válido y se produce una violación de integridad referencial.

Paso 8: Ejecutar el proyecto

```
dotnet run
```

dotnet run → compila y ejecuta el proyecto.

Resultado esperado: aparecen las secciones con los resultados de cada demostración. Se observa el tiempo de SingleQuery y SplitQuery, y el SQL generado por cada técnica.

Paso 9: Analizar la salida

La salida del programa muestra información como la siguiente:

```
=== SPLIT QUERIES ===

--- Single Query (producto cartesiano) ---
Órdenes: 3 | Tiempo: 55 ms
  OF-2024-0001: 3 planchas | 2 aleaciones
  OF-2024-0002: 2 planchas | 1 aleaciones
  OF-2024-0003: 1 planchas | 1 aleaciones

--- Split Query (sin producto cartesiano) ---
Órdenes: 3 | Tiempo: 48 ms
  OF-2024-0001: 3 planchas | 2 aleaciones
  OF-2024-0002: 2 planchas | 1 aleaciones
  OF-2024-0003: 1 planchas | 1 aleaciones

--- Comparación de rendimiento ---
Single Query: 55 ms | Órdenes: 3
Split Query: 48 ms | Órdenes: 3

--- SQL de Single Query ---
SELECT [o].[Id], [o].[NumeroOrden], ..., [p].[Id], [p].[Espesor], ..., [oa].[OrdenFabricacionId], [oa].[AleacionId], ..., [a].[Id], [a].[Nombre], ...
FROM [OrdenesFabricacion] AS [o]
LEFT JOIN [PlanchasAcero] AS [p] ON [o].[Id] = [p].[OrdenId]
LEFT JOIN [OrdenesAleaciones] AS [oa] ON [o].[Id] = [oa].[OrdenFabricacionId]
LEFT JOIN [Aleaciones] AS [a] ON [oa].[AleacionId] = [a].[Id]
WHERE [o].[IsDeleted] = CAST(0 AS bit) AND [p].[IsDeleted] = CAST(0 AS bit)
ORDER BY [o].[Id], [p].[Id], [oa].[OrdenFabricacionId], [oa].[AleacionId]
```



```

--- SQL de Split Query ---
-- Consulta 1: órdenes
SELECT [o].[Id], [o].[NumeroOrden], ...
FROM [OrdenesFabricacion] AS [o]
WHERE [o].[IsDeleted] = CAST(0 AS bit)
ORDER BY [o].[Id]

-- Consulta 2: planchas
SELECT [p].[Id], [p].[OrdenId], [p].[Espesor], ..., [o].[Id]
FROM [OrdenesFabricacion] AS [o]
INNER JOIN [PlanchasAcero] AS [p] ON [o].[Id] = [p].[OrdenId]
WHERE [o].[IsDeleted] = CAST(0 AS bit) AND [p].[IsDeleted] = CAST(0 AS bit)
ORDER BY [o].[Id], [p].[Id]

-- Consulta 3: entidades intermedias con aleaciones
SELECT [oa].[OrdenFabricacionId], [oa].[AleacionId], ..., [a].[Id], [a].[Nombre], ..., [o].[Id]
FROM [OrdenesFabricacion] AS [o]
INNER JOIN [OrdenesAleaciones] AS [oa] ON [o].[Id] = [oa].[OrdenFabricacionId]
INNER JOIN [Aleaciones] AS [a] ON [oa].[AleacionId] = [a].[Id]
WHERE [o].[IsDeleted] = CAST(0 AS bit)
ORDER BY [o].[Id], [oa].[OrdenFabricacionId], [oa].[AleacionId]

```

La primera sección muestra el tiempo de SingleQuery. La segunda sección muestra el tiempo de SplitQuery. La tercera sección muestra la comparación. La cuarta sección muestra el SQL de SingleQuery. La quinta sección muestra el SQL de SplitQuery.

Observaciones: SplitQuery es ligeramente más rápida porque no genera el producto cartesiano. El SQL de SingleQuery incluye tres LEFT JOIN con todas las columnas. El SQL de SplitQuery incluye tres consultas separadas. El número de filas transferidas es menor en SplitQuery.

Paso 10: Diagnosticar un error común

Modificar el método ObtenerConVariasColeccionesSingleQuery para eliminar AsNoTrackingWithIdentityResolution y usar AsNoTracking:

```

public List<OrdenFabricacion> ObtenerConVariasColeccionesSingleQuery()
{
    return _context.OrdenesFabricacion
        .AsNoTracking()
        .Include(o => o.Planchas)
        .Include(o => o.OrdenesAleaciones)
        .ThenInclude(oa => oa.Aleacion)
        .AsSingleQuery()
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}

```

Resultado esperado: el código funciona, pero las entidades relacionadas pueden aparecer como instancias duplicadas porque AsNoTracking no usa la caché de identidad. Las planchas y las aleaciones que aparecen en varias órdenes se instancian varias veces.

Solución: usar AsNoTrackingWithIdentityResolution para garantizar instancias únicas.

```

public List<OrdenFabricacion> ObtenerConVariasColeccionesSingleQuery()
{
    return _context.OrdenesFabricacion
        .AsNoTrackingWithIdentityResolution()
        .Include(o => o.Planchas)
        .Include(o => o.OrdenesAleaciones)
        .ThenInclude(oa => oa.Aleacion)
        .AsSingleQuery()
        .OrderBy(o => o.NumeroOrden)
        .ToList();
}

```

Resultado esperado con la solución: las entidades relacionadas se resuelven a instancias únicas.

Errores comunes del ejercicio

Error	Causa	Solución
Producto cartesiano	Se incluyeron varias colecciones sin AsSplitQuery	Usar AsSplitQuery
Instancias duplicadas	Se usó AsNoTracking con relaciones	Usar AsNoTrackingWithIdentityResolution
AsSplitQuery con una colección	No aporta beneficio	Usar AsSingleQuery
Coherencia de datos	Cada consulta se ejecuta en una transacción separada	Usar transacción explícita
Comportamiento por defecto	Se configuró SplitQuery globalmente	Revisar el comportamiento por defecto
Advertencia de producto cartesiano	Se ignoró la advertencia	Configurar ConfigureWarnings

Reto resuelto: Configurar el comportamiento por defecto de Split Queries

Reto: Configurar el DbContext para que el comportamiento por defecto sea SplitQuery cuando se incluyen varias colecciones. Configurar la advertencia de producto cartesiano como error. Verificar el comportamiento con una consulta con dos colecciones.

Solución paso a paso:

Paso 1: Modificar el método OnConfiguring del AceriaDbContext:

```
protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
{
    if (!optionsBuilder.IsConfigured)
    {
        optionsBuilder
            .UseSqlServer(
                "Server=(localdb)\\mssqllocaldb;Database=AceriaDB;Trusted_Connection=True;",
                sqlOptions =>
                {
                    sqlOptions.EnableRetryOnFailure(maxRetryCount: 5);
                    sqlOptions.CommandTimeout(60);
                    sqlOptions.UseQuerySplittingBehavior(QuerySplittingBehavior.SplitQuery);
                })
            .ConfigureWarnings(warnings =>
                warnings.Throw(RelationalEventId.MultipleCollectionIncludeWarning));
    }
}
```

Línea 1: protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder) → método de configuración.

Línea 3: if (!optionsBuilder.IsConfigured) → comprueba si las opciones ya están configuradas.

Línea 5: optionsBuilder → objeto de configuración.

Línea 6: .UseSqlServer(→ registra el proveedor de SQL Server.

Línea 7: "Server=(localdb)\\mssqllocaldb;Database=AceriaDB;Trusted_Connection=True;", → cadena de conexión.

Línea 8: sqlOptions => → delegado de configuración.

Línea 10: sqlOptions.EnableRetryOnFailure(maxRetryCount: 5); → habilita los reintentos.

Línea 11: `sqlOptions.CommandTimeout(60);` → establece el tiempo de espera.

Línea 12: `sqlOptions.UseQuerySplittingBehavior(QuerySplittingBehavior.SplitQuery);` → establece `SplitQuery` como comportamiento por defecto.

Línea 13: `}` → cierra el delegado.

Línea 14: `.ConfigureWarnings(warnings =>)` → configura las advertencias.

Línea 15: `warnings.Throw(RelationalEventId.MultipleCollectionIncludeWarning);` → convierte la advertencia en error.

Línea 16: `}` → cierra el bloque.

Paso 2: Ejecutar una consulta con dos colecciones sin `AsSplitQuery`:

```
var ordenes = context.OrdenesFabricacion
    .Include(o => o.Planchas)
    .Include(o => o.OrdenesAleaciones)
    .ToList();
```

Resultado esperado: EF Core lanza una excepción `InvalidOperationException` indicando que la consulta incluye varias colecciones sin `AsSplitQuery`.

Solución: añadir `AsSplitQuery` explícitamente.

```
var ordenes = context.OrdenesFabricacion
    .Include(o => o.Planchas)
    .Include(o => o.OrdenesAleaciones)
    .AsSplitQuery()
    .ToList();
```

Resultado esperado con la solución: la consulta se ejecuta sin error porque `AsSplitQuery` está aplicado explícitamente.

Paso 3: Ejecutar `dotnet run` y verificar que la consulta con `AsSplitQuery` funciona y que la consulta sin `AsSplitQuery` lanza la excepción.

Resultado esperado: el comportamiento por defecto es `SplitQuery` y la advertencia de producto cartesiano se convierte en error.

Analogía final

Las Split Queries en una acería son como la forma de pedir las carpetas al archivo central. Si se piden todas las carpetas de planchas y todas las carpetas de aleaciones en un solo viaje, el archivo tiene que combinar cada plancha con cada aleación, generando un montón enorme de documentos. Si una orden tiene tres planchas y dos aleaciones, el archivo trae seis documentos combinados. Si tiene cien planchas y cien aleaciones, trae diez mil documentos combinados. La Split Query es como pedir en tres viajes separados: uno para las órdenes, otro para las planchas y otro para las aleaciones. El archivo trae cien planchas y cien aleaciones, no diez mil combinaciones. Cada viaje es más pequeño y más rápido. La desventaja es que los datos pueden cambiar entre un viaje y el siguiente. Para evitarlo, se hace todo dentro de una misma transacción: el archivo bloquea las carpetas hasta que se termina de pedir todo. La configuración global es como la política del archivo: por defecto, pedir en varios viajes cuando hay varias colecciones. La advertencia de producto cartesiano es como un aviso del archivo: si se pide todo en un solo viaje y hay varias colecciones, el archivo avisa. Así funcionan las Split Queries en EF Core: se eligen según el escenario y se configuran según las necesidades.

Resultado esperado

Al final del ejercicio, deberías haber:

Añadido los métodos de Split Queries a la interfaz `IOrdenRepositorio`.

Implementado los métodos en `OrdenRepositorio`.

Creado el caso de uso `SplitQueriesUseCase`.

Registrado el caso de uso en el contenedor.

Insertado datos de prueba con órdenes, planchas y aleaciones.

Ejecutado las demostraciones de `AsSingleQuery` y `AsSplitQuery`.

Comparado el rendimiento de ambas técnicas.

Analizado el SQL generado por cada técnica.

Diagnosticado el error de usar `AsNoTracking` en lugar de `AsNoTrackingWithIdentityResolution`.

Configurado el comportamiento por defecto de Split Queries y la advertencia de producto cartesiano.
